

Section 81 Editorial — Master Graph Decomposition Mixed Contest

A. Unique Sink Population

How to find it

Every eventual universal destination must lie in a sink SCC; it exists exactly when that sink is unique.

Correctness

From any SCC that can be the eventual destination for all vertices, no condensation edge may leave it; otherwise following that edge reaches a different class that cannot return. If two sinks exist, neither reaches the other. If exactly one sink exists, every condensation-DAG path extended until it stops ends there, so every vertex reaches precisely that component.

Complexity

$O(n+m)$ time and memory.

C++

```
#include <bits/stdc++.h>
using namespace std;
struct SCC {
    int n, c = 0;
    vector<vector<int>> g, rg;
    vector<int> id, order;
    SCC(int n, vector<pair<int, int>> e) : n(n), g(n), rg(n), id(n, -1) {
        for (auto [u, v] : e)
            g[u].push_back(v), rg[v].push_back(u);
        vector<char> seen(n);
        for (int s = 0; s < n; s++)
            if (!seen[s]) {
                vector<pair<int, int>> st = {{s, 0}};
                seen[s] = 1;
                while (!st.empty()) {
                    auto &[u, i] = st.back();
                    if (i < (int)g[u].size()) {
                        int v = g[u][i++];
                        if (!seen[v])
                            seen[v] = 1, st.push_back({v, 0});
                    } else
                        order.push_back(u), st.pop_back();
                }
            }
        reverse(order.begin(), order.end());
        for (int s : order)
            if (id[s] < 0) {
                id[s] = c;
                vector<int> st = {s};
                while (!st.empty()) {
                    int u = st.back();
```

```

        st.pop_back();
        for (int v : rg[u])
            if (id[v] < 0)
                id[v] = c, st.push_back(v);
    }
    c++;
}

};

struct LowLink {
    int n, timer = 0;
    vector<pair<int, int>> e;
    vector<vector<pair<int, int>>> g;
    vector<int> tin, low, sub, parent_edge, it;
    vector<char> bridge;
    vector<vector<int>> parts;
    LowLink(int n, vector<pair<int, int>> e)
        : n(n), e(e), g(g(n), tin(n, -1), low(n), sub(n), parent_edge(n, -1), it(n),
          bridge(e.size()), parts(n) {
        for (int i = 0; i < (int)e.size(); i++) {
            auto [u, v] = e[i];
            g[u].push_back({v, i});
            g[v].push_back({u, i});
        }
        tin[0] = low[0] = timer++;
        sub[0] = 1;
        vector<int> st = {0};
        while (!st.empty()) {
            int u = st.back();
            if (it[u] < (int)g[u].size()) {
                auto [v, id] = g[u][it[u]++];
                if (id == parent_edge[u])
                    continue;
                if (tin[v] >= 0)
                    low[u] = min(low[u], tin[v]);
                else {
                    parent_edge[v] = id;
                    tin[v] = low[v] = timer++;
                    sub[v] = 1;
                    st.push_back(v);
                }
            } else {
                st.pop_back();
                int id = parent_edge[u];
                if (id < 0)
                    continue;
                auto [a, b] = e[id];
                int p = a ^ b ^ u;
                sub[p] += sub[u];
                low[p] = min(low[p], low[u]);
                if (low[u] > tin[p])
                    bridge[id] = 1;
                if (low[u] >= tin[p])
                    parts[p].push_back(sub[u]);
            }
        }
    }
};

pair<vector<int>, vector<vector<int>>>
bridge_tree(int n, const vector<pair<int, int>> &e) {
    LowLink l(n, e);
    vector<int> id(n, -1);
    int c = 0;

```

```

for (int s = 0; s < n; s++)
    if (id[s] < 0) {
        id[s] = c;
        vector<int> st = {s};
        while (!st.empty()) {
            int u = st.back();
            st.pop_back();
            for (auto [v, k] : l.g[u])
                if (!l.bridge[k] && id[v] < 0)
                    id[v] = c, st.push_back(v);
        }
        c++;
    }
vector<vector<int>> t(c);
for (int i = 0; i < (int)e.size(); i++)
    if (l.bridge[i]) {
        auto [u, v] = e[i];
        u = id[u];
        v = id[v];
        t[u].push_back(v);
        t[v].push_back(u);
    }
return {id, t};
}

struct TreeDistance {
    int n, L;
    vector<int> d;
    vector<vector<int>> up;
    TreeDistance(vector<vector<int>> t)
        : n(t.size()), L(max(1, (int)bit_width((unsigned)max(1, n)))), d(n),
          up(L, vector<int>(n)) {
        vector<pair<int, int>> st = {{0, 0}};
        while (!st.empty()) {
            auto [u, p] = st.back();
            st.pop_back();
            up[0][u] = p;
            for (int v : t[u])
                if (v != p)
                    d[v] = d[u] + 1, st.push_back({v, u});
        }
        for (int j = 1; j < L; j++)
            for (int i = 0; i < n; i++)
                up[j][i] = up[j - 1][up[j - 1][i]];
    }

    int lca(int a, int b) {
        if (d[a] < d[b])
            swap(a, b);
        int z = d[a] - d[b];
        for (int j = 0; j < L; j++)
            if (z >> j & 1)
                a = up[j][a];
        if (a == b)
            return a;
        for (int j = L - 1; j >= 0; j--)
            if (up[j][a] != up[j][b])
                a = up[j][a], b = up[j][b];
        return up[0][a];
    }

    int dist(int a, int b) {
        int c = lca(a, b);
        return d[a] + d[b] - 2 * d[c];
    }
};

```

```

vector<vector<int>> block_cut(int n, const vector<pair<int, int>> &e) {
    vector<vector<pair<int, int>>> g(n);
    for (int i = 0; i < (int)e.size(); i++) {
        auto [u, v] = e[i];
        g[u].push_back({v, i});
        g[v].push_back({u, i});
    }
    vector<int> tin(n, -1), low(n), edges, parent(n, -1), it(n);
    vector<vector<int>> t(n);
    int timer = 0;
    tin[0] = low[0] = timer++;
    vector<int> dfs = {0};
    while (!dfs.empty()) {
        int u = dfs.back();
        if (it[u] < (int)g[u].size()) {
            auto [v, id] = g[u][it[u]++];
            if (id == parent[u])
                continue;
            if (tin[v] < 0) {
                parent[v] = id;
                edges.push_back(id);
                tin[v] = low[v] = timer++;
                dfs.push_back(v);
            } else if (tin[v] < tin[u])
                edges.push_back(id), low[u] = min(low[u], tin[v]);
        } else {
            dfs.pop_back();
            int id = parent[u];
            if (id < 0)
                continue;
            auto [a, b] = e[id];
            int p = a ^ b ^ u;
            low[p] = min(low[p], low[u]);
            if (low[u] >= tin[p]) {
                vector<int> vs;
                while (1) {
                    int x = edges.back();
                    edges.pop_back();
                    vs.push_back(e[x].first);
                    vs.push_back(e[x].second);
                    if (x == id)
                        break;
                }
                sort(vs.begin(), vs.end());
                vs.erase(unique(vs.begin(), vs.end()), vs.end());
                int block = t.size();
                t.push_back({});
                for (int x : vs)
                    t[block].push_back(x), t[x].push_back(block);
            }
        }
    }
    return t;
}

vector<bitset<1500>> dominators(int n, const vector<pair<int, int>> &e) {
    vector<vector<int>> g(n), pred(n);
    for (auto [u, v] : e)
        g[u].push_back(v), pred[v].push_back(u);
    bitset<1500> reach;
    reach[0] = 1;
    vector<int> st = {0};
    while (!st.empty()) {
        int u = st.back();

```

```

    st.pop_back();
    for (int v : g[u])
        if (!reach[v])
            reach[v] = 1, st.push_back(v);
}
vector<bitset<1500>> d(n, reach);
d[0].reset();
d[0][0] = 1;
bool change = 1;
while (change) {
    change = 0;
    for (int v = 1; v < n; v++)
        if (reach[v]) {
            bitset<1500> x = reach;
            for (int p : pred[v])
                if (reach[p])
                    x &= d[p];
            x[v] = 1;
            if (x != d[v])
                d[v] = x, change = 1;
        }
}
return d;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> e(m);
    for (auto &[u, v] : e)
        cin >> u >> v, --u, --v;
    SCC s(n, e);
    vector<int> size(s.c);
    vector<char> out(s.c);
    for (int v = 0; v < n; v++)
        size[s.id[v]]++;
    for (auto [u, v] : e)
        if (s.id[u] != s.id[v])
            out[s.id[u]] = 1;
    int sink = -1;
    for (int c = 0; c < s.c; c++)
        if (!out[c]) {
            if (sink != -1) {
                cout << 0 << '\n';
                return 0;
            }
            sink = c;
        }
    cout << size[sink] << '\n';
}

```

Python

```

import sys

sys.setrecursionlimit(1_000_000)

def scc(n, edges):
    graph = [[] for _ in range(n)]

```

```

reverse = [[] for _ in range(n)]
for u, v in edges:
    graph[u].append(v)
    reverse[v].append(u)
seen = [False] * n
order = []
for start in range(n):
    if seen[start]:
        continue
    stack = [(start, 0)]
    seen[start] = True
    while stack:
        node, index = stack[-1]
        if index < len(graph[node]):
            target = graph[node][index]
            stack[-1] = (node, index + 1)
            if not seen[target]:
                seen[target] = True
                stack.append((target, 0))
        else:
            order.append(node)
            stack.pop()
component = [-1] * n
count = 0
for start in reversed(order):
    if component[start] != -1:
        continue
    component[start] = count
    stack = [start]
    while stack:
        node = stack.pop()
        for target in reverse[node]:
            if component[target] == -1:
                component[target] = count
                stack.append(target)
    count += 1
return component, count

def undirected_lowlink(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    subtree = [0] * n
    bridges = [False] * len(edges)
    separating = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        subtree[node] = 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:
                continue
            if tin[target] != -1:
                low[node] = min(low[node], tin[target])
            else:
                dfs(target, edge_id)

```

```

        subtree[node] += subtree[target]
        low[node] = min(low[node], low[target])
        if low[target] > tin[node]:
            bridges[edge_id] = True
        if low[target] >= tin[node]:
            separating[node].append(subtree[target])

dfs(0)
return graph, tin, low, subtree, bridges, separating

def bridge_tree(n, edges):
    graph, _, _, _, bridges, _ = undirected_lowlink(n, edges)
    component = [-1] * n
    count = 0
    for start in range(n):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for target, edge_id in graph[node]:
                if not bridges[edge_id] and component[target] == -1:
                    component[target] = count
                    stack.append(target)
        count += 1
    tree = [[] for _ in range(count)]
    for edge_id, (u, v) in enumerate(edges):
        if bridges[edge_id]:
            a, b = component[u], component[v]
            tree[a].append(b)
            tree[b].append(a)
    return component, tree

def tree_lca(tree):
    size = len(tree)
    levels = max(1, size.bit_length())
    up = [[0] * size for _ in range(levels)]
    depth = [0] * size
    stack = [(0, 0)]
    order = [0]
    while stack:
        node, parent = stack.pop()
        up[0][node] = parent
        for target in tree[node]:
            if target != parent:
                depth[target] = depth[node] + 1
                order.append(target)
                stack.append((target, node))
    for level in range(1, levels):
        previous = up[level - 1]
        up[level] = [previous[previous[node]] for node in range(size)]

def lca(a, b):
    if depth[a] < depth[b]:
        a, b = b, a
    difference = depth[a] - depth[b]
    for level in range(levels):
        if difference >> level & 1:
            a = up[level][a]
    if a == b:

```

```

        return a
    for level in range(levels - 1, -1, -1):
        if up[level][a] != up[level][b]:
            a, b = up[level][a], up[level][b]
    return up[0][a]

def distance(a, b):
    ancestor = lca(a, b)
    return depth[a] + depth[b] - 2 * depth[ancestor]

return distance

def block_cut_tree(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    edge_stack = []
    tree = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:
                continue
            if tin[target] == -1:
                edge_stack.append(edge_id)
                dfs(target, edge_id)
                low[node] = min(low[node], low[target])
                if low[target] >= tin[node]:
                    vertices = set()
                    while True:
                        taken = edge_stack.pop()
                        vertices.update(edges[taken])
                        if taken == edge_id:
                            break
                    block = len(tree)
                    tree.append([])
                    for vertex in vertices:
                        tree[block].append(vertex)
                        tree[vertex].append(block)
                elif tin[target] < tin[node]:
                    edge_stack.append(edge_id)
                    low[node] = min(low[node], tin[target])

    dfs(0)
    return tree

def dominators(n, edges):
    predecessors = [[] for _ in range(n)]
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        predecessors[v].append(u)
    reachable = 1
    stack = [0]

```

```

while stack:
    node = stack.pop()
    for target in graph[node]:
        if not (reachable >> target & 1):
            reachable |= 1 << target
            stack.append(target)
dom = [reachable] * n
dom[0] = 1
changed = True
while changed:
    changed = False
    for node in range(1, n):
        if not (reachable >> node & 1):
            continue
        value = reachable
        for parent in predecessors[node]:
            if reachable >> parent & 1:
                value &= dom[parent]
        value |= 1 << node
        if value != dom[node]:
            dom[node] = value
            changed = True
return dom

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[:2]
    edges = [(data[i] - 1, data[i + 1] - 1) for i in range(2, 2 + 2 * m, 2)]
    component, count = scc(n, edges)
    out = [0] * count
    size = [0] * count
    for v in range(n):
        size[component[v]] += 1
    for u, v in edges:
        if component[u] != component[v]:
            out[component[u]] = 1
    sinks = [c for c in range(count) if not out[c]]
    print(size[sinks[0]] if len(sinks) == 1 else 0)

if __name__ == "__main__":
    main()

```

B. Strong Connectivity Repairs

How to find it

Unless already strongly connected, each condensation source needs an incoming edge and each sink an outgoing edge; cyclic pairing attains the maximum of the counts.

Correctness

Every source SCC needs a new incoming edge and every sink SCC needs a new outgoing edge, so at least the larger count is necessary. List sources and sinks cyclically (repeating entries from the shorter list) and connect each sink to the next source. The added cycle links every old DAG region into one strongly connected tour, attaining the bound.

Complexity

$O(n+m)$ time and memory.

C++

```
#include <bits/stdc++.h>
using namespace std;
struct SCC {
    int n, c = 0;
    vector<vector<int>> g, rg;
    vector<int> id, order;
    SCC(int n, vector<pair<int, int>> e) : n(n), g(n), rg(n), id(n, -1) {
        for (auto [u, v] : e)
            g[u].push_back(v), rg[v].push_back(u);
        vector<char> seen(n);
        for (int s = 0; s < n; s++)
            if (!seen[s]) {
                vector<pair<int, int>> st = {{s, 0}};
                seen[s] = 1;
                while (!st.empty()) {
                    auto &[u, i] = st.back();
                    if (i < (int)g[u].size()) {
                        int v = g[u][i++];
                        if (!seen[v])
                            seen[v] = 1, st.push_back({v, 0});
                    } else
                        order.push_back(u), st.pop_back();
                }
            }
        reverse(order.begin(), order.end());
        for (int s : order)
            if (id[s] < 0) {
                id[s] = c;
                vector<int> st = {s};
                while (!st.empty()) {
                    int u = st.back();
                    st.pop_back();
                    for (int v : rg[u])
                        if (id[v] < 0)
                            id[v] = c, st.push_back(v);
                }
                c++;
            }
    }
};

struct LowLink {
    int n, timer = 0;
    vector<pair<int, int>> e;
    vector<vector<pair<int, int>>> g;
    vector<int> tin, low, sub, parent_edge, it;
    vector<char> bridge;
    vector<vector<int>> parts;
    LowLink(int n, vector<pair<int, int>> e)
        : n(n), e(e), g(n), tin(n, -1), low(n), sub(n), parent_edge(n, -1), it(n),
          bridge(e.size()), parts(n) {
        for (int i = 0; i < (int)e.size(); i++) {
            auto [u, v] = e[i];
            g[u].push_back({v, i});
            g[v].push_back({u, i});
        }
        tin[0] = low[0] = timer++;
        sub[0] = 1;
        vector<int> st = {0};
        while (!st.empty()) {
            int u = st.back();
            if (it[u] < (int)g[u].size()) {

```

```

        auto [v, id] = g[u][it[u]++];
        if (id == parent_edge[u])
            continue;
        if (tin[v] >= 0)
            low[u] = min(low[u], tin[v]);
        else {
            parent_edge[v] = id;
            tin[v] = low[v] = timer++;
            sub[v] = 1;
            st.push_back(v);
        }
    } else {
        st.pop_back();
        int id = parent_edge[u];
        if (id < 0)
            continue;
        auto [a, b] = e[id];
        int p = a ^ b ^ u;
        sub[p] += sub[u];
        low[p] = min(low[p], low[u]);
        if (low[u] > tin[p])
            bridge[id] = 1;
        if (low[u] >= tin[p])
            parts[p].push_back(sub[u]);
    }
}
}
};

pair<vector<int>, vector<vector<int>>>
bridge_tree(int n, const vector<pair<int, int>> &e) {
    LowLink l(n, e);
    vector<int> id(n, -1);
    int c = 0;
    for (int s = 0; s < n; s++)
        if (id[s] < 0) {
            id[s] = c;
            vector<int> st = {s};
            while (!st.empty()) {
                int u = st.back();
                st.pop_back();
                for (auto [v, k] : l.g[u])
                    if (!l.bridge[k] && id[v] < 0)
                        id[v] = c, st.push_back(v);
            }
            c++;
        }
    vector<vector<int>> t(c);
    for (int i = 0; i < (int)e.size(); i++)
        if (l.bridge[i]) {
            auto [u, v] = e[i];
            u = id[u];
            v = id[v];
            t[u].push_back(v);
            t[v].push_back(u);
        }
    return {id, t};
}

struct TreeDistance {
    int n, L;
    vector<int> d;
    vector<vector<int>> up;
    TreeDistance(vector<vector<int>> t)
        : n(t.size()), L(max(1, (int)bit_width((unsigned)max(1, n)))), d(n),

```

```

        up(L, vector<int>(n)) {
vector<pair<int, int>> st = {{0, 0}};
while (!st.empty()) {
    auto [u, p] = st.back();
    st.pop_back();
    up[0][u] = p;
    for (int v : t[u])
        if (v != p)
            d[v] = d[u] + 1, st.push_back({v, u});
}
for (int j = 1; j < L; j++)
    for (int i = 0; i < n; i++)
        up[j][i] = up[j - 1][up[j - 1][i]];
}

int lca(int a, int b) {
    if (d[a] < d[b])
        swap(a, b);
    int z = d[a] - d[b];
    for (int j = 0; j < L; j++)
        if (z >> j & 1)
            a = up[j][a];
    if (a == b)
        return a;
    for (int j = L - 1; j >= 0; j--)
        if (up[j][a] != up[j][b])
            a = up[j][a], b = up[j][b];
    return up[0][a];
}

int dist(int a, int b) {
    int c = lca(a, b);
    return d[a] + d[b] - 2 * d[c];
}
};

vector<vector<int>> block_cut(int n, const vector<pair<int, int>> &e) {
    vector<vector<pair<int, int>>> g(n);
    for (int i = 0; i < (int)e.size(); i++) {
        auto [u, v] = e[i];
        g[u].push_back({v, i});
        g[v].push_back({u, i});
    }
    vector<int> tin(n, -1), low(n), edges, parent(n, -1), it(n);
    vector<vector<int>> t(n);
    int timer = 0;
    tin[0] = low[0] = timer++;
    vector<int> dfs = {0};
    while (!dfs.empty()) {
        int u = dfs.back();
        if (it[u] < (int)g[u].size()) {
            auto [v, id] = g[u][it[u]++];
            if (id == parent[u])
                continue;
            if (tin[v] < 0) {
                parent[v] = id;
                edges.push_back(id);
                tin[v] = low[v] = timer++;
                dfs.push_back(v);
            } else if (tin[v] < tin[u])
                edges.push_back(id), low[u] = min(low[u], tin[v]);
        } else {
            dfs.pop_back();
            int id = parent[u];
            if (id < 0)
                continue;

```

```

    auto [a, b] = e[id];
    int p = a ^ b ^ u;
    low[p] = min(low[p], low[u]);
    if (low[u] >= tin[p]) {
        vector<int> vs;
        while (1) {
            int x = edges.back();
            edges.pop_back();
            vs.push_back(e[x].first);
            vs.push_back(e[x].second);
            if (x == id)
                break;
        }
        sort(vs.begin(), vs.end());
        vs.erase(unique(vs.begin(), vs.end()), vs.end());
        int block = t.size();
        t.push_back({});
        for (int x : vs)
            t[block].push_back(x), t[x].push_back(block);
    }
}
}
return t;
}
vector<bitset<1500>> dominators(int n, const vector<pair<int, int>> &e) {
    vector<vector<int>> g(n), pred(n);
    for (auto [u, v] : e)
        g[u].push_back(v), pred[v].push_back(u);
    bitset<1500> reach;
    reach[0] = 1;
    vector<int> st = {0};
    while (!st.empty()) {
        int u = st.back();
        st.pop_back();
        for (int v : g[u])
            if (!reach[v])
                reach[v] = 1, st.push_back(v);
    }
    vector<bitset<1500>> d(n, reach);
    d[0].reset();
    d[0][0] = 1;
    bool change = 1;
    while (change) {
        change = 0;
        for (int v = 1; v < n; v++)
            if (reach[v]) {
                bitset<1500> x = reach;
                for (int p : pred[v])
                    if (reach[p])
                        x &= d[p];
                x[v] = 1;
                if (x != d[v])
                    d[v] = x, change = 1;
            }
    }
    return d;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> e(m);

```

```

for (auto &[u, v] : e)
    cin >> u >> v, --u, --v;
SCC s(n, e);
if (s.c == 1) {
    cout << 0 << '\n';
    return 0;
}
vector<char> in(s.c), out(s.c);
for (auto [u, v] : e)
    if (s.id[u] != s.id[v])
        out[s.id[u]] = in[s.id[v]] = 1;
cout << max(count(in.begin(), in.end(), 0), count(out.begin(), out.end(), 0))
    << '\n';
}

```

Python

```

import sys

sys.setrecursionlimit(1_000_000)

def scc(n, edges):
    graph = [[] for _ in range(n)]
    reverse = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        reverse[v].append(u)
    seen = [False] * n
    order = []
    for start in range(n):
        if seen[start]:
            continue
        stack = [(start, 0)]
        seen[start] = True
        while stack:
            node, index = stack[-1]
            if index < len(graph[node]):
                target = graph[node][index]
                stack[-1] = (node, index + 1)
                if not seen[target]:
                    seen[target] = True
                    stack.append((target, 0))
            else:
                order.append(node)
                stack.pop()
    component = [-1] * n
    count = 0
    for start in reversed(order):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for target in reverse[node]:
                if component[target] == -1:
                    component[target] = count
                    stack.append(target)
        count += 1
    return component, count

```

```

def undirected_lowlink(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    subtree = [0] * n
    bridges = [False] * len(edges)
    separating = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        subtree[node] = 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:
                continue
            if tin[target] != -1:
                low[node] = min(low[node], tin[target])
            else:
                dfs(target, edge_id)
                subtree[node] += subtree[target]
                low[node] = min(low[node], low[target])
                if low[target] > tin[node]:
                    bridges[edge_id] = True
                if low[target] >= tin[node]:
                    separating[node].append(subtree[target])

    dfs(0)
    return graph, tin, low, subtree, bridges, separating

def bridge_tree(n, edges):
    graph, _, _, _, bridges, _ = undirected_lowlink(n, edges)
    component = [-1] * n
    count = 0
    for start in range(n):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for target, edge_id in graph[node]:
                if not bridges[edge_id] and component[target] == -1:
                    component[target] = count
                    stack.append(target)
        count += 1
    tree = [[] for _ in range(count)]
    for edge_id, (u, v) in enumerate(edges):
        if bridges[edge_id]:
            a, b = component[u], component[v]
            tree[a].append(b)
            tree[b].append(a)
    return component, tree

def tree_lca(tree):
    size = len(tree)

```

```

levels = max(1, size.bit_length())
up = [[0] * size for _ in range(levels)]
depth = [0] * size
stack = [(0, 0)]
order = [0]
while stack:
    node, parent = stack.pop()
    up[0][node] = parent
    for target in tree[node]:
        if target != parent:
            depth[target] = depth[node] + 1
            order.append(target)
            stack.append((target, node))
for level in range(1, levels):
    previous = up[level - 1]
    up[level] = [previous[previous[node]] for node in range(size)]

def lca(a, b):
    if depth[a] < depth[b]:
        a, b = b, a
    difference = depth[a] - depth[b]
    for level in range(levels):
        if difference >> level & 1:
            a = up[level][a]
    if a == b:
        return a
    for level in range(levels - 1, -1, -1):
        if up[level][a] != up[level][b]:
            a, b = up[level][a], up[level][b]
    return up[0][a]

def distance(a, b):
    ancestor = lca(a, b)
    return depth[a] + depth[b] - 2 * depth[ancestor]

return distance

def block_cut_tree(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    edge_stack = []
    tree = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:
                continue
            if tin[target] == -1:
                edge_stack.append(edge_id)
                dfs(target, edge_id)
                low[node] = min(low[node], low[target])
            if low[target] >= tin[node]:
                vertices = set()
                while True:

```

```

        taken = edge_stack.pop()
        vertices.update(edges[taken])
        if taken == edge_id:
            break
        block = len(tree)
        tree.append([])
        for vertex in vertices:
            tree[block].append(vertex)
            tree[vertex].append(block)
    elif tin[target] < tin[node]:
        edge_stack.append(edge_id)
        low[node] = min(low[node], tin[target])

dfs(0)
return tree

def dominators(n, edges):
    predecessors = [[] for _ in range(n)]
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        predecessors[v].append(u)
    reachable = 1
    stack = [0]
    while stack:
        node = stack.pop()
        for target in graph[node]:
            if not (reachable >> target & 1):
                reachable |= 1 << target
                stack.append(target)
    dom = [reachable] * n
    dom[0] = 1
    changed = True
    while changed:
        changed = False
        for node in range(1, n):
            if not (reachable >> node & 1):
                continue
            value = reachable
            for parent in predecessors[node]:
                if reachable >> parent & 1:
                    value &= dom[parent]
            value |= 1 << node
            if value != dom[node]:
                dom[node] = value
                changed = True
    return dom

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[:2]
    edges = [(data[i] - 1, data[i + 1] - 1) for i in range(2, 2 + 2 * m, 2)]
    component, count = scc(n, edges)
    if count == 1:
        print(0)
        return
    inside = [0] * count
    outside = [0] * count
    for u, v in edges:
        if component[u] != component[v]:
            outside[component[u]] = inside[component[v]] = 1

```

```

    print(max(inside.count(0), outside.count(0)))

if __name__ == "__main__":
    main()

```

C. Eulerian Word Chain

How to find it

Treat each word as a labeled edge and run lexicographic Hierholzer after checking Euler degree conditions.

Correctness

The first and last letters make each word one distinct directed edge, so a valid ordering is exactly an Euler trail. Hierholzer consumes every word once and produces a valid chain after reversal. If fewer than n words return, another used-edge component was unreachable. Sorting labeled edges makes the first differing word in the resulting valid itinerary minimal.

Complexity

$O(n+m)$ time and memory.

C++

```

#include <bits/stdc++.h>
using namespace std;
struct SCC {
    int n, c = 0;
    vector<vector<int>> g, rg;
    vector<int> id, order;
    SCC(int n, vector<pair<int, int>> e) : n(n), g(n), rg(n), id(n, -1) {
        for (auto [u, v] : e)
            g[u].push_back(v), rg[v].push_back(u);
        vector<char> seen(n);
        for (int s = 0; s < n; s++)
            if (!seen[s]) {
                vector<pair<int, int>> st = {{s, 0}};
                seen[s] = 1;
                while (!st.empty()) {
                    auto &[u, i] = st.back();
                    if (i < (int)g[u].size()) {
                        int v = g[u][i++];
                        if (!seen[v])
                            seen[v] = 1, st.push_back({v, 0});
                    } else
                        order.push_back(u), st.pop_back();
                }
            }
        reverse(order.begin(), order.end());
        for (int s : order)
            if (id[s] < 0) {
                id[s] = c;
                vector<int> st = {s};
                while (!st.empty()) {
                    int u = st.back();
                    st.pop_back();
                    for (int v : rg[u])
                        if (id[v] < 0)

```

```

        id[v] = c, st.push_back(v);
    }
    c++;
}
}
};

struct LowLink {
    int n, timer = 0;
    vector<pair<int, int>> e;
    vector<vector<pair<int, int>>> g;
    vector<int> tin, low, sub, parent_edge, it;
    vector<char> bridge;
    vector<vector<int>> parts;
    LowLink(int n, vector<pair<int, int>> e)
        : n(n), e(e), g(n), tin(n, -1), low(n), sub(n), parent_edge(n, -1), it(n),
          bridge(e.size()), parts(n) {
        for (int i = 0; i < (int)e.size(); i++) {
            auto [u, v] = e[i];
            g[u].push_back({v, i});
            g[v].push_back({u, i});
        }
        tin[0] = low[0] = timer++;
        sub[0] = 1;
        vector<int> st = {0};
        while (!st.empty()) {
            int u = st.back();
            if (it[u] < (int)g[u].size()) {
                auto [v, id] = g[u][it[u]++];
                if (id == parent_edge[u])
                    continue;
                if (tin[v] >= 0)
                    low[u] = min(low[u], tin[v]);
                else {
                    parent_edge[v] = id;
                    tin[v] = low[v] = timer++;
                    sub[v] = 1;
                    st.push_back(v);
                }
            } else {
                st.pop_back();
                int id = parent_edge[u];
                if (id < 0)
                    continue;
                auto [a, b] = e[id];
                int p = a ^ b ^ u;
                sub[p] += sub[u];
                low[p] = min(low[p], low[u]);
                if (low[u] > tin[p])
                    bridge[id] = 1;
                if (low[u] >= tin[p])
                    parts[p].push_back(sub[u]);
            }
        }
    }
};

pair<vector<int>, vector<vector<int>>>
bridge_tree(int n, const vector<pair<int, int>> &e) {
    LowLink l(n, e);
    vector<int> id(n, -1);
    int c = 0;
    for (int s = 0; s < n; s++)
        if (id[s] < 0) {
            id[s] = c;

```

```

    vector<int> st = {s};
    while (!st.empty()) {
        int u = st.back();
        st.pop_back();
        for (auto [v, k] : l.g[u])
            if (!l.bridge[k] && id[v] < 0)
                id[v] = c, st.push_back(v);
    }
    c++;
}

vector<vector<int>> t(c);
for (int i = 0; i < (int)e.size(); i++)
    if (l.bridge[i]) {
        auto [u, v] = e[i];
        u = id[u];
        v = id[v];
        t[u].push_back(v);
        t[v].push_back(u);
    }
return {id, t};
}

struct TreeDistance {
    int n, L;
    vector<int> d;
    vector<vector<int>> up;
    TreeDistance(vector<vector<int>> t)
        : n(t.size()), L(max(1, (int)bit_width((unsigned)max(1, n)))), d(n),
          up(L, vector<int>(n)) {
        vector<pair<int, int>> st = {{0, 0}};
        while (!st.empty()) {
            auto [u, p] = st.back();
            st.pop_back();
            up[0][u] = p;
            for (int v : t[u])
                if (v != p)
                    d[v] = d[u] + 1, st.push_back({v, u});
        }
        for (int j = 1; j < L; j++)
            for (int i = 0; i < n; i++)
                up[j][i] = up[j - 1][up[j - 1][i]];
    }

    int lca(int a, int b) {
        if (d[a] < d[b])
            swap(a, b);
        int z = d[a] - d[b];
        for (int j = 0; j < L; j++)
            if (z >> j & 1)
                a = up[j][a];
        if (a == b)
            return a;
        for (int j = L - 1; j >= 0; j--)
            if (up[j][a] != up[j][b])
                a = up[j][a], b = up[j][b];
        return up[0][a];
    }

    int dist(int a, int b) {
        int c = lca(a, b);
        return d[a] + d[b] - 2 * d[c];
    }
};

vector<vector<int>> block_cut(int n, const vector<pair<int, int>> &e) {
    vector<vector<pair<int, int>>> g(n);
    for (int i = 0; i < (int)e.size(); i++) {

```

```

    auto [u, v] = e[i];
    g[u].push_back({v, i});
    g[v].push_back({u, i});
}
vector<int> tin(n, -1), low(n), edges, parent(n, -1), it(n);
vector<vector<int>> t(n);
int timer = 0;
tin[0] = low[0] = timer++;
vector<int> dfs = {0};
while (!dfs.empty()) {
    int u = dfs.back();
    if (it[u] < (int)g[u].size()) {
        auto [v, id] = g[u][it[u]++];
        if (id == parent[u])
            continue;
        if (tin[v] < 0) {
            parent[v] = id;
            edges.push_back(id);
            tin[v] = low[v] = timer++;
            dfs.push_back(v);
        } else if (tin[v] < tin[u])
            edges.push_back(id), low[u] = min(low[u], tin[v]);
    } else {
        dfs.pop_back();
        int id = parent[u];
        if (id < 0)
            continue;
        auto [a, b] = e[id];
        int p = a ^ b ^ u;
        low[p] = min(low[p], low[u]);
        if (low[u] >= tin[p]) {
            vector<int> vs;
            while (1) {
                int x = edges.back();
                edges.pop_back();
                vs.push_back(e[x].first);
                vs.push_back(e[x].second);
                if (x == id)
                    break;
            }
            sort(vs.begin(), vs.end());
            vs.erase(unique(vs.begin(), vs.end()), vs.end());
            int block = t.size();
            t.push_back({});
            for (int x : vs)
                t[block].push_back(x), t[x].push_back(block);
        }
    }
}
return t;
}
vector<bitset<1500>> dominators(int n, const vector<pair<int, int>> &e) {
    vector<vector<int>> g(n), pred(n);
    for (auto [u, v] : e)
        g[u].push_back(v), pred[v].push_back(u);
    bitset<1500> reach;
    reach[0] = 1;
    vector<int> st = {0};
    while (!st.empty()) {
        int u = st.back();
        st.pop_back();
        for (int v : g[u])
            if (!reach[v])

```

```

        reach[v] = 1, st.push_back(v);
    }
    vector<bitset<1500>> d(n, reach);
    d[0].reset();
    d[0][0] = 1;
    bool change = 1;
    while (change) {
        change = 0;
        for (int v = 1; v < n; v++)
            if (reach[v]) {
                bitset<1500> x = reach;
                for (int p : pred[v])
                    if (reach[p])
                        x &= d[p];
                x[v] = 1;
                if (x != d[v])
                    d[v] = x, change = 1;
            }
    }
    return d;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<vector<pair<string, int>>> g(26);
    vector<int> in(26), out(26);
    for (int i = 0; i < n; i++) {
        string w;
        cin >> w;
        int u = w.front() - 'a', v = w.back() - 'a';
        g[u].push_back({w, v});
        out[u]++;
        in[v]++;
    }
    vector<int> plus, minus;
    for (int v = 0; v < 26; v++) {
        if (out[v] - in[v] == 1)
            plus.push_back(v);
        else if (in[v] - out[v] == 1)
            minus.push_back(v);
        else if (in[v] != out[v]) {
            cout << "IMPOSSIBLE\n";
            return 0;
        }
    }
    if (plus.size() != minus.size() || plus.size() > 1) {
        cout << "IMPOSSIBLE\n";
        return 0;
    }
    int start = plus.empty() ? -1 : plus[0];
    if (start < 0)
        for (int i = 0; i < 26; i++)
            if (out[i]) {
                start = i;
                break;
            }
    for (auto &v : g)
        sort(v.rbegin(), v.rend());
    vector<pair<int, string>> st = {{start, ""}};
    vector<string> ans;
    while (!st.empty()) {

```

```

    int u = st.back().first;
    if (g[u].empty()) {
        if (!st.back().second.empty())
            ans.push_back(st.back().second);
        st.pop_back();
    } else {
        auto [w, v] = g[u].back();
        g[u].pop_back();
        st.push_back({v, w});
    }
}
}
if ((int)ans.size() != n)
    cout << "IMPOSSIBLE\n";
else {
    reverse(ans.begin(), ans.end());
    for (int i = 0; i < n; i++)
        cout << ans[i] << " \n"[i + 1 == n];
}
}

```

Python

```

import sys

sys.setrecursionlimit(1_000_000)

def scc(n, edges):
    graph = [[] for _ in range(n)]
    reverse = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        reverse[v].append(u)
    seen = [False] * n
    order = []
    for start in range(n):
        if seen[start]:
            continue
        stack = [(start, 0)]
        seen[start] = True
        while stack:
            node, index = stack[-1]
            if index < len(graph[node]):
                target = graph[node][index]
                stack[-1] = (node, index + 1)
                if not seen[target]:
                    seen[target] = True
                    stack.append((target, 0))
            else:
                order.append(node)
                stack.pop()
    component = [-1] * n
    count = 0
    for start in reversed(order):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for target in reverse[node]:

```

```

        if component[target] == -1:
            component[target] = count
            stack.append(target)

    count += 1
    return component, count

def undirected_lowlink(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    subtree = [0] * n
    bridges = [False] * len(edges)
    separating = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        subtree[node] = 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:
                continue
            if tin[target] != -1:
                low[node] = min(low[node], tin[target])
            else:
                dfs(target, edge_id)
                subtree[node] += subtree[target]
                low[node] = min(low[node], low[target])
                if low[target] > tin[node]:
                    bridges[edge_id] = True
                if low[target] >= tin[node]:
                    separating[node].append(subtree[target])

    dfs(0)
    return graph, tin, low, subtree, bridges, separating

def bridge_tree(n, edges):
    graph, _, _, _, bridges, _ = undirected_lowlink(n, edges)
    component = [-1] * n
    count = 0
    for start in range(n):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for target, edge_id in graph[node]:
                if not bridges[edge_id] and component[target] == -1:
                    component[target] = count
                    stack.append(target)

        count += 1
    tree = [[] for _ in range(count)]
    for edge_id, (u, v) in enumerate(edges):
        if bridges[edge_id]:
            a, b = component[u], component[v]
            tree[a].append(b)

```

```

        tree[b].append(a)
    return component, tree

def tree_lca(tree):
    size = len(tree)
    levels = max(1, size.bit_length())
    up = [[0] * size for _ in range(levels)]
    depth = [0] * size
    stack = [(0, 0)]
    order = [0]
    while stack:
        node, parent = stack.pop()
        up[0][node] = parent
        for target in tree[node]:
            if target != parent:
                depth[target] = depth[node] + 1
                order.append(target)
                stack.append((target, node))
    for level in range(1, levels):
        previous = up[level - 1]
        up[level] = [previous[previous[node]] for node in range(size)]

    def lca(a, b):
        if depth[a] < depth[b]:
            a, b = b, a
        difference = depth[a] - depth[b]
        for level in range(levels):
            if difference >> level & 1:
                a = up[level][a]
        if a == b:
            return a
        for level in range(levels - 1, -1, -1):
            if up[level][a] != up[level][b]:
                a, b = up[level][a], up[level][b]
        return up[0][a]

    def distance(a, b):
        ancestor = lca(a, b)
        return depth[a] + depth[b] - 2 * depth[ancestor]

    return distance

def block_cut_tree(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    edge_stack = []
    tree = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:
                continue
            if tin[target] == -1:

```

```

        edge_stack.append(edge_id)
        dfs(target, edge_id)
        low[node] = min(low[node], low[target])
        if low[target] >= tin[node]:
            vertices = set()
            while True:
                taken = edge_stack.pop()
                vertices.update(edges[taken])
                if taken == edge_id:
                    break
            block = len(tree)
            tree.append([])
            for vertex in vertices:
                tree[block].append(vertex)
                tree[vertex].append(block)
        elif tin[target] < tin[node]:
            edge_stack.append(edge_id)
            low[node] = min(low[node], tin[target])

    dfs(0)
    return tree

def dominators(n, edges):
    predecessors = [[] for _ in range(n)]
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        predecessors[v].append(u)
    reachable = 1
    stack = [0]
    while stack:
        node = stack.pop()
        for target in graph[node]:
            if not (reachable >> target & 1):
                reachable |= 1 << target
                stack.append(target)
    dom = [reachable] * n
    dom[0] = 1
    changed = True
    while changed:
        changed = False
        for node in range(1, n):
            if not (reachable >> node & 1):
                continue
            value = reachable
            for parent in predecessors[node]:
                if reachable >> parent & 1:
                    value &= dom[parent]
            value |= 1 << node
            if value != dom[node]:
                dom[node] = value
                changed = True
    return dom

def main():
    words = sys.stdin.buffer.read().decode().split()
    n = int(words[0])
    words = words[1:]
    graph = [[] for _ in range(26)]
    indeg = [0] * 26
    out = [0] * 26

```

```

for word in words:
    u = ord(word[0]) - 97
    v = ord(word[-1]) - 97
    graph[u].append((word, v))
    out[u] += 1
    indeg[v] += 1
starts = [v for v in range(26) if out[v] == indeg[v] + 1]
ends = [v for v in range(26) if indeg[v] == out[v] + 1]
if len(starts) > 1 or len(ends) > 1 or len(starts) != len(ends):
    print("IMPOSSIBLE")
    return
start = starts[0] if starts else next((v for v in range(26) if out[v]), 0)
for row in graph:
    row.sort(reverse=True)
stack = [(start, None)]
answer = []
while stack:
    v, _ = stack[-1]
    if graph[v]:
        word, to = graph[v].pop()
        stack.append((to, word))
    else:
        _, word = stack.pop()
        if word is not None:
            answer.append(word)
if len(answer) != n:
    print("IMPOSSIBLE")
else:
    print(*reversed(answer))

if __name__ == "__main__":
    main()

```

D. Robbins Orientation

How to find it

A bridge can never lie on a directed cycle, while DFS back edges orient every bridgeless tree edge into a strongly connected orientation.

Correctness

A bridge cannot receive a strongly connected orientation because whichever direction it receives blocks reachability across it in the reverse direction. Conversely, in a connected bridgeless graph orient DFS tree edges downward and back edges toward ancestors. Every subtree has a back-edge escape above its parent edge, and induction gives directed routes both to and from the root; thus all vertices are mutually reachable.

Complexity

$O(n+m)$ time and memory.

C++

```

#include <bits/stdc++.h>
using namespace std;
struct SCC {
    int n, c = 0;

```

```

vector<vector<int>> g, rg;
vector<int> id, order;
SCC(int n, vector<pair<int, int>> e) : n(n), g(n), rg(n), id(n, -1) {
    for (auto [u, v] : e)
        g[u].push_back(v), rg[v].push_back(u);
    vector<char> seen(n);
    for (int s = 0; s < n; s++)
        if (!seen[s]) {
            vector<pair<int, int>> st = {{s, 0}};
            seen[s] = 1;
            while (!st.empty()) {
                auto &[u, i] = st.back();
                if (i < (int)g[u].size()) {
                    int v = g[u][i++];
                    if (!seen[v])
                        seen[v] = 1, st.push_back({v, 0});
                } else
                    order.push_back(u), st.pop_back();
            }
        }
    reverse(order.begin(), order.end());
    for (int s : order)
        if (id[s] < 0) {
            id[s] = c;
            vector<int> st = {s};
            while (!st.empty()) {
                int u = st.back();
                st.pop_back();
                for (int v : rg[u])
                    if (id[v] < 0)
                        id[v] = c, st.push_back(v);
            }
            c++;
        }
    }
};

struct LowLink {
    int n, timer = 0;
    vector<pair<int, int>> e;
    vector<vector<pair<int, int>>> g;
    vector<int> tin, low, sub, parent_edge, it;
    vector<char> bridge;
    vector<vector<int>> parts;
    LowLink(int n, vector<pair<int, int>> e)
        : n(n), e(e), g(n), tin(n, -1), low(n), sub(n), parent_edge(n, -1), it(n),
          bridge(e.size()), parts(n) {
        for (int i = 0; i < (int)e.size(); i++) {
            auto [u, v] = e[i];
            g[u].push_back({v, i});
            g[v].push_back({u, i});
        }
        tin[0] = low[0] = timer++;
        sub[0] = 1;
        vector<int> st = {0};
        while (!st.empty()) {
            int u = st.back();
            if (it[u] < (int)g[u].size()) {
                auto [v, id] = g[u][it[u]++];
                if (id == parent_edge[u])
                    continue;
                if (tin[v] >= 0)
                    low[u] = min(low[u], tin[v]);
                else {

```

```

        parent_edge[v] = id;
        tin[v] = low[v] = timer++;
        sub[v] = 1;
        st.push_back(v);
    }
} else {
    st.pop_back();
    int id = parent_edge[u];
    if (id < 0)
        continue;
    auto [a, b] = e[id];
    int p = a ^ b ^ u;
    sub[p] += sub[u];
    low[p] = min(low[p], low[u]);
    if (low[u] > tin[p])
        bridge[id] = 1;
    if (low[u] >= tin[p])
        parts[p].push_back(sub[u]);
}
}
}
};

pair<vector<int>, vector<vector<int>>>
bridge_tree(int n, const vector<pair<int, int>> &e) {
    LowLink l(n, e);
    vector<int> id(n, -1);
    int c = 0;
    for (int s = 0; s < n; s++)
        if (id[s] < 0) {
            id[s] = c;
            vector<int> st = {s};
            while (!st.empty()) {
                int u = st.back();
                st.pop_back();
                for (auto [v, k] : l.g[u])
                    if (!l.bridge[k] && id[v] < 0)
                        id[v] = c, st.push_back(v);
            }
            c++;
        }
    vector<vector<int>> t(c);
    for (int i = 0; i < (int)e.size(); i++)
        if (l.bridge[i]) {
            auto [u, v] = e[i];
            u = id[u];
            v = id[v];
            t[u].push_back(v);
            t[v].push_back(u);
        }
    return {id, t};
}

struct TreeDistance {
    int n, L;
    vector<int> d;
    vector<vector<int>> up;
    TreeDistance(vector<vector<int>> t)
        : n(t.size()), L(max(1, (int)bit_width((unsigned)max(1, n)))), d(n),
          up(L, vector<int>(n)) {
        vector<pair<int, int>> st = {{0, 0}};
        while (!st.empty()) {
            auto [u, p] = st.back();
            st.pop_back();
            up[0][u] = p;

```

```

    for (int v : t[u])
        if (v != p)
            d[v] = d[u] + 1, st.push_back({v, u});
}
for (int j = 1; j < L; j++)
    for (int i = 0; i < n; i++)
        up[j][i] = up[j - 1][up[j - 1][i]];
}
int lca(int a, int b) {
    if (d[a] < d[b])
        swap(a, b);
    int z = d[a] - d[b];
    for (int j = 0; j < L; j++)
        if (z >> j & 1)
            a = up[j][a];
    if (a == b)
        return a;
    for (int j = L - 1; j >= 0; j--)
        if (up[j][a] != up[j][b])
            a = up[j][a], b = up[j][b];
    return up[0][a];
}
int dist(int a, int b) {
    int c = lca(a, b);
    return d[a] + d[b] - 2 * d[c];
}
};

vector<vector<int>>> block_cut(int n, const vector<pair<int, int>>> &e) {
    vector<vector<pair<int, int>>> g(n);
    for (int i = 0; i < (int)e.size(); i++) {
        auto [u, v] = e[i];
        g[u].push_back({v, i});
        g[v].push_back({u, i});
    }
    vector<int> tin(n, -1), low(n), edges, parent(n, -1), it(n);
    vector<vector<int>>> t(n);
    int timer = 0;
    tin[0] = low[0] = timer++;
    vector<int> dfs = {0};
    while (!dfs.empty()) {
        int u = dfs.back();
        if (it[u] < (int)g[u].size()) {
            auto [v, id] = g[u][it[u]++];
            if (id == parent[u])
                continue;
            if (tin[v] < 0) {
                parent[v] = id;
                edges.push_back(id);
                tin[v] = low[v] = timer++;
                dfs.push_back(v);
            } else if (tin[v] < tin[u])
                edges.push_back(id), low[u] = min(low[u], tin[v]);
        } else {
            dfs.pop_back();
            int id = parent[u];
            if (id < 0)
                continue;
            auto [a, b] = e[id];
            int p = a ^ b ^ u;
            low[p] = min(low[p], low[u]);
            if (low[u] >= tin[p]) {
                vector<int> vs;
                while (1) {

```

```

        int x = edges.back();
        edges.pop_back();
        vs.push_back(e[x].first);
        vs.push_back(e[x].second);
        if (x == id)
            break;
    }
    sort(vs.begin(), vs.end());
    vs.erase(unique(vs.begin(), vs.end()), vs.end());
    int block = t.size();
    t.push_back({});
    for (int x : vs)
        t[block].push_back(x), t[x].push_back(block);
    }
}
}
return t;
}
}

vector<bitset<1500>> dominators(int n, const vector<pair<int, int>> &e) {
    vector<vector<int>> g(n), pred(n);
    for (auto [u, v] : e)
        g[u].push_back(v), pred[v].push_back(u);
    bitset<1500> reach;
    reach[0] = 1;
    vector<int> st = {0};
    while (!st.empty()) {
        int u = st.back();
        st.pop_back();
        for (int v : g[u])
            if (!reach[v])
                reach[v] = 1, st.push_back(v);
    }
    vector<bitset<1500>> d(n, reach);
    d[0].reset();
    d[0][0] = 1;
    bool change = 1;
    while (change) {
        change = 0;
        for (int v = 1; v < n; v++)
            if (reach[v]) {
                bitset<1500> x = reach;
                for (int p : pred[v])
                    if (reach[p])
                        x &= d[p];
                x[v] = 1;
                if (x != d[v])
                    d[v] = x, change = 1;
            }
    }
    return d;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> e(m);
    for (auto &[u, v] : e)
        cin >> u >> v, --u, --v;
    LowLink l(n, e);
    cout << (count(l.bridge.begin(), l.bridge.end(), 1) ? "NO\n" : "YES\n");
}

```

Python

```
import sys

sys.setrecursionlimit(1_000_000)

def scc(n, edges):
    graph = [[] for _ in range(n)]
    reverse = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        reverse[v].append(u)
    seen = [False] * n
    order = []
    for start in range(n):
        if seen[start]:
            continue
        stack = [(start, 0)]
        seen[start] = True
        while stack:
            node, index = stack[-1]
            if index < len(graph[node]):
                target = graph[node][index]
                stack[-1] = (node, index + 1)
                if not seen[target]:
                    seen[target] = True
                    stack.append((target, 0))
            else:
                order.append(node)
                stack.pop()
    component = [-1] * n
    count = 0
    for start in reversed(order):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for target in reverse[node]:
                if component[target] == -1:
                    component[target] = count
                    stack.append(target)
        count += 1
    return component, count

def undirected_lowlink(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    subtree = [0] * n
    bridges = [False] * len(edges)
    separating = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
```

```

    timer += 1
    subtree[node] = 1
    for target, edge_id in graph[node]:
        if edge_id == parent_edge:
            continue
        if tin[target] != -1:
            low[node] = min(low[node], tin[target])
        else:
            dfs(target, edge_id)
            subtree[node] += subtree[target]
            low[node] = min(low[node], low[target])
            if low[target] > tin[node]:
                bridges[edge_id] = True
            if low[target] >= tin[node]:
                separating[node].append(subtree[target])

    dfs(0)
    return graph, tin, low, subtree, bridges, separating

def bridge_tree(n, edges):
    graph, _, _, _, bridges, _ = undirected_lowlink(n, edges)
    component = [-1] * n
    count = 0
    for start in range(n):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for target, edge_id in graph[node]:
                if not bridges[edge_id] and component[target] == -1:
                    component[target] = count
                    stack.append(target)
        count += 1
    tree = [[] for _ in range(count)]
    for edge_id, (u, v) in enumerate(edges):
        if bridges[edge_id]:
            a, b = component[u], component[v]
            tree[a].append(b)
            tree[b].append(a)
    return component, tree

def tree_lca(tree):
    size = len(tree)
    levels = max(1, size.bit_length())
    up = [[0] * size for _ in range(levels)]
    depth = [0] * size
    stack = [(0, 0)]
    order = [0]
    while stack:
        node, parent = stack.pop()
        up[0][node] = parent
        for target in tree[node]:
            if target != parent:
                depth[target] = depth[node] + 1
                order.append(target)
                stack.append((target, node))
    for level in range(1, levels):
        previous = up[level - 1]
        up[level] = [previous[previous[node]] for node in range(size)]

```

```

def lca(a, b):
    if depth[a] < depth[b]:
        a, b = b, a
    difference = depth[a] - depth[b]
    for level in range(levels):
        if difference >> level & 1:
            a = up[level][a]
    if a == b:
        return a
    for level in range(levels - 1, -1, -1):
        if up[level][a] != up[level][b]:
            a, b = up[level][a], up[level][b]
    return up[0][a]

def distance(a, b):
    ancestor = lca(a, b)
    return depth[a] + depth[b] - 2 * depth[ancestor]

return distance

def block_cut_tree(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    edge_stack = []
    tree = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:
                continue
            if tin[target] == -1:
                edge_stack.append(edge_id)
                dfs(target, edge_id)
                low[node] = min(low[node], low[target])
                if low[target] >= tin[node]:
                    vertices = set()
                    while True:
                        taken = edge_stack.pop()
                        vertices.update(edges[taken])
                        if taken == edge_id:
                            break
                    block = len(vertices)
                    tree.append([])
                    for vertex in vertices:
                        tree[block].append(vertex)
                        tree[vertex].append(block)
            elif tin[target] < tin[node]:
                edge_stack.append(edge_id)
                low[node] = min(low[node], tin[target])

    dfs(0)
    return tree

```

```

def dominators(n, edges):
    predecessors = [[] for _ in range(n)]
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        predecessors[v].append(u)
    reachable = 1
    stack = [0]
    while stack:
        node = stack.pop()
        for target in graph[node]:
            if not (reachable >> target & 1):
                reachable |= 1 << target
                stack.append(target)
    dom = [reachable] * n
    dom[0] = 1
    changed = True
    while changed:
        changed = False
        for node in range(1, n):
            if not (reachable >> node & 1):
                continue
            value = reachable
            for parent in predecessors[node]:
                if reachable >> parent & 1:
                    value &= dom[parent]
            value |= 1 << node
            if value != dom[node]:
                dom[node] = value
                changed = True
    return dom

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[:2]
    edges = [(data[i] - 1, data[i + 1] - 1) for i in range(2, 2 + 2 * m, 2)]
    *, bridges, _ = undirected_lowlink(n, edges)
    print("NO" if any(bridges) else "YES")

if __name__ == "__main__":
    main()

```

E. Failed Vertex Routes

How to find it

Removing c disconnects u and v exactly when c lies on every original u - v path, including the endpoint cases.

Correctness

When the failed vertex is an endpoint, connection is impossible. Otherwise u and v disconnect after removing c exactly when every old path used c . Section 80's block-cut characterization tests this all-path property exactly, so negating it yields precisely the surviving-connectivity answer.

Complexity

$O((n+m) \log n + q \log n)$ time and $O((n+m) \log n)$ memory.

C++

```
#include <bits/stdc++.h>
using namespace std;
struct SCC {
    int n, c = 0;
    vector<vector<int>> g, rg;
    vector<int> id, order;
    SCC(int n, vector<pair<int, int>> e) : n(n), g(n), rg(n), id(n, -1) {
        for (auto [u, v] : e)
            g[u].push_back(v), rg[v].push_back(u);
        vector<char> seen(n);
        for (int s = 0; s < n; s++)
            if (!seen[s]) {
                vector<pair<int, int>> st = {{s, 0}};
                seen[s] = 1;
                while (!st.empty()) {
                    auto &[u, i] = st.back();
                    if (i < (int)g[u].size()) {
                        int v = g[u][i++];
                        if (!seen[v])
                            seen[v] = 1, st.push_back({v, 0});
                    } else
                        order.push_back(u), st.pop_back();
                }
            }
        reverse(order.begin(), order.end());
        for (int s : order)
            if (id[s] < 0) {
                id[s] = c;
                vector<int> st = {s};
                while (!st.empty()) {
                    int u = st.back();
                    st.pop_back();
                    for (int v : rg[u])
                        if (id[v] < 0)
                            id[v] = c, st.push_back(v);
                }
                c++;
            }
    }
};

struct LowLink {
    int n, timer = 0;
    vector<pair<int, int>> e;
    vector<vector<pair<int, int>>> g;
    vector<int> tin, low, sub, parent_edge, it;
    vector<char> bridge;
    vector<vector<int>> parts;
    LowLink(int n, vector<pair<int, int>> e)
        : n(n), e(e), g(n), tin(n, -1), low(n), sub(n), parent_edge(n, -1), it(n),
          bridge(e.size()), parts(n) {
        for (int i = 0; i < (int)e.size(); i++) {
            auto [u, v] = e[i];
            g[u].push_back({v, i});
            g[v].push_back({u, i});
        }
        tin[0] = low[0] = timer++;
        sub[0] = 1;
        vector<int> st = {0};
        while (!st.empty()) {
            int u = st.back();
            if (it[u] < (int)g[u].size()) {

```

```

        auto [v, id] = g[u][it[u]++];
        if (id == parent_edge[u])
            continue;
        if (tin[v] >= 0)
            low[u] = min(low[u], tin[v]);
        else {
            parent_edge[v] = id;
            tin[v] = low[v] = timer++;
            sub[v] = 1;
            st.push_back(v);
        }
    } else {
        st.pop_back();
        int id = parent_edge[u];
        if (id < 0)
            continue;
        auto [a, b] = e[id];
        int p = a ^ b ^ u;
        sub[p] += sub[u];
        low[p] = min(low[p], low[u]);
        if (low[u] > tin[p])
            bridge[id] = 1;
        if (low[u] >= tin[p])
            parts[p].push_back(sub[u]);
    }
}
}
};

pair<vector<int>, vector<vector<int>>>>
bridge_tree(int n, const vector<pair<int, int>> &e) {
    LowLink l(n, e);
    vector<int> id(n, -1);
    int c = 0;
    for (int s = 0; s < n; s++)
        if (id[s] < 0) {
            id[s] = c;
            vector<int> st = {s};
            while (!st.empty()) {
                int u = st.back();
                st.pop_back();
                for (auto [v, k] : l.g[u])
                    if (!l.bridge[k] && id[v] < 0)
                        id[v] = c, st.push_back(v);
            }
            c++;
        }
    vector<vector<int>> t(c);
    for (int i = 0; i < (int)e.size(); i++)
        if (l.bridge[i]) {
            auto [u, v] = e[i];
            u = id[u];
            v = id[v];
            t[u].push_back(v);
            t[v].push_back(u);
        }
    return {id, t};
}

struct TreeDistance {
    int n, L;
    vector<int> d;
    vector<vector<int>> up;
    TreeDistance(vector<vector<int>> t)
        : n(t.size()), L(max(1, (int)bit_width((unsigned)max(1, n)))), d(n),

```

```

        up(L, vector<int>(n)) {
vector<pair<int, int>> st = {{0, 0}};
while (!st.empty()) {
    auto [u, p] = st.back();
    st.pop_back();
    up[0][u] = p;
    for (int v : t[u])
        if (v != p)
            d[v] = d[u] + 1, st.push_back({v, u});
}
for (int j = 1; j < L; j++)
    for (int i = 0; i < n; i++)
        up[j][i] = up[j - 1][up[j - 1][i]];
}

int lca(int a, int b) {
    if (d[a] < d[b])
        swap(a, b);
    int z = d[a] - d[b];
    for (int j = 0; j < L; j++)
        if (z >> j & 1)
            a = up[j][a];
    if (a == b)
        return a;
    for (int j = L - 1; j >= 0; j--)
        if (up[j][a] != up[j][b])
            a = up[j][a], b = up[j][b];
    return up[0][a];
}

int dist(int a, int b) {
    int c = lca(a, b);
    return d[a] + d[b] - 2 * d[c];
}
};

vector<vector<int>> block_cut(int n, const vector<pair<int, int>> &e) {
    vector<vector<pair<int, int>>> g(n);
    for (int i = 0; i < (int)e.size(); i++) {
        auto [u, v] = e[i];
        g[u].push_back({v, i});
        g[v].push_back({u, i});
    }
    vector<int> tin(n, -1), low(n), edges, parent(n, -1), it(n);
    vector<vector<int>> t(n);
    int timer = 0;
    tin[0] = low[0] = timer++;
    vector<int> dfs = {0};
    while (!dfs.empty()) {
        int u = dfs.back();
        if (it[u] < (int)g[u].size()) {
            auto [v, id] = g[u][it[u]++];
            if (id == parent[u])
                continue;
            if (tin[v] < 0) {
                parent[v] = id;
                edges.push_back(id);
                tin[v] = low[v] = timer++;
                dfs.push_back(v);
            } else if (tin[v] < tin[u])
                edges.push_back(id), low[u] = min(low[u], tin[v]);
        } else {
            dfs.pop_back();
            int id = parent[u];
            if (id < 0)
                continue;

```

```

    auto [a, b] = e[id];
    int p = a ^ b ^ u;
    low[p] = min(low[p], low[u]);
    if (low[u] >= tin[p]) {
        vector<int> vs;
        while (1) {
            int x = edges.back();
            edges.pop_back();
            vs.push_back(e[x].first);
            vs.push_back(e[x].second);
            if (x == id)
                break;
        }
        sort(vs.begin(), vs.end());
        vs.erase(unique(vs.begin(), vs.end()), vs.end());
        int block = t.size();
        t.push_back({});
        for (int x : vs)
            t[block].push_back(x), t[x].push_back(block);
    }
}
}
return t;
}
vector<bitset<1500>> dominators(int n, const vector<pair<int, int>> &e) {
    vector<vector<int>> g(n), pred(n);
    for (auto [u, v] : e)
        g[u].push_back(v), pred[v].push_back(u);
    bitset<1500> reach;
    reach[0] = 1;
    vector<int> st = {0};
    while (!st.empty()) {
        int u = st.back();
        st.pop_back();
        for (int v : g[u])
            if (!reach[v])
                reach[v] = 1, st.push_back(v);
    }
    vector<bitset<1500>> d(n, reach);
    d[0].reset();
    d[0][0] = 1;
    bool change = 1;
    while (change) {
        change = 0;
        for (int v = 1; v < n; v++)
            if (reach[v]) {
                bitset<1500> x = reach;
                for (int p : pred[v])
                    if (reach[p])
                        x &= d[p];
                x[v] = 1;
                if (x != d[v])
                    d[v] = x, change = 1;
            }
    }
    return d;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    vector<pair<int, int>> e(m);

```

```

for (auto &[u, v] : e)
    cin >> u >> v, --u, --v;
TreeDistance d(block_cut(n, e));
while (q--) {
    int u, v, c;
    cin >> u >> v >> c;
    --u;
    --v;
    --c;
    bool bad = c == u || c == v || d.dist(u, v) == d.dist(u, c) + d.dist(c, v);
    cout << (bad ? "NO\n" : "YES\n");
}
}

```

Python

```

import sys

sys.setrecursionlimit(1_000_000)

def scc(n, edges):
    graph = [[] for _ in range(n)]
    reverse = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        reverse[v].append(u)
    seen = [False] * n
    order = []
    for start in range(n):
        if seen[start]:
            continue
        stack = [(start, 0)]
        seen[start] = True
        while stack:
            node, index = stack[-1]
            if index < len(graph[node]):
                target = graph[node][index]
                stack[-1] = (node, index + 1)
                if not seen[target]:
                    seen[target] = True
                    stack.append((target, 0))
            else:
                order.append(node)
                stack.pop()
    component = [-1] * n
    count = 0
    for start in reversed(order):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for target in reverse[node]:
                if component[target] == -1:
                    component[target] = count
                    stack.append(target)
        count += 1
    return component, count

```

```

def undirected_lowlink(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    subtree = [0] * n
    bridges = [False] * len(edges)
    separating = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        subtree[node] = 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:
                continue
            if tin[target] != -1:
                low[node] = min(low[node], tin[target])
            else:
                dfs(target, edge_id)
                subtree[node] += subtree[target]
                low[node] = min(low[node], low[target])
                if low[target] > tin[node]:
                    bridges[edge_id] = True
                if low[target] >= tin[node]:
                    separating[node].append(subtree[target])

    dfs(0)
    return graph, tin, low, subtree, bridges, separating

def bridge_tree(n, edges):
    graph, _, _, bridges, _ = undirected_lowlink(n, edges)
    component = [-1] * n
    count = 0
    for start in range(n):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for target, edge_id in graph[node]:
                if not bridges[edge_id] and component[target] == -1:
                    component[target] = count
                    stack.append(target)
        count += 1
    tree = [[] for _ in range(count)]
    for edge_id, (u, v) in enumerate(edges):
        if bridges[edge_id]:
            a, b = component[u], component[v]
            tree[a].append(b)
            tree[b].append(a)
    return component, tree

def tree_lca(tree):
    size = len(tree)
    levels = max(1, size.bit_length())

```

```

up = [[0] * size for _ in range(levels)]
depth = [0] * size
stack = [(0, 0)]
order = [0]
while stack:
    node, parent = stack.pop()
    up[0][node] = parent
    for target in tree[node]:
        if target != parent:
            depth[target] = depth[node] + 1
            order.append(target)
            stack.append((target, node))
for level in range(1, levels):
    previous = up[level - 1]
    up[level] = [previous[previous[node]] for node in range(size)]

def lca(a, b):
    if depth[a] < depth[b]:
        a, b = b, a
    difference = depth[a] - depth[b]
    for level in range(levels):
        if difference >> level & 1:
            a = up[level][a]
    if a == b:
        return a
    for level in range(levels - 1, -1, -1):
        if up[level][a] != up[level][b]:
            a, b = up[level][a], up[level][b]
    return up[0][a]

def distance(a, b):
    ancestor = lca(a, b)
    return depth[a] + depth[b] - 2 * depth[ancestor]

return distance

def block_cut_tree(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    edge_stack = []
    tree = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:
                continue
            if tin[target] == -1:
                edge_stack.append(edge_id)
                dfs(target, edge_id)
                low[node] = min(low[node], low[target])
                if low[target] >= tin[node]:
                    vertices = set()
                    while True:
                        taken = edge_stack.pop()

```

```

        vertices.update(edges[taken])
        if taken == edge_id:
            break
        block = len(tree)
        tree.append([])
        for vertex in vertices:
            tree[block].append(vertex)
            tree[vertex].append(block)
    elif tin[target] < tin[node]:
        edge_stack.append(edge_id)
        low[node] = min(low[node], tin[target])

dfs(0)
return tree

def dominators(n, edges):
    predecessors = [[] for _ in range(n)]
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        predecessors[v].append(u)
    reachable = 1
    stack = [0]
    while stack:
        node = stack.pop()
        for target in graph[node]:
            if not (reachable >> target & 1):
                reachable |= 1 << target
                stack.append(target)
    dom = [reachable] * n
    dom[0] = 1
    changed = True
    while changed:
        changed = False
        for node in range(1, n):
            if not (reachable >> node & 1):
                continue
            value = reachable
            for parent in predecessors[node]:
                if reachable >> parent & 1:
                    value &= dom[parent]
            value |= 1 << node
            if value != dom[node]:
                dom[node] = value
                changed = True
    return dom

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m, q = data[:3]
    at = 3
    edges = [(data[i] - 1, data[i + 1] - 1) for i in range(at, at + 2 * m, 2)]
    at += 2 * m
    distance = tree_lca(block_cut_tree(n, edges))
    out = []
    for i in range(at, at + 3 * q, 3):
        u, v, c = data[i] - 1, data[i + 1] - 1, data[i + 2] - 1
        bad = c in (u, v) or distance(u, v) == distance(u, c) + distance(c, v)
        out.append("NO" if bad else "YES")
    print("\n".join(out))

```

```
if __name__ == "__main__":
    main()
```

F. Dominator Subtree Queries

How to find it

The fixed-point equation $\text{Dom}[v] = \{v\} \cup \bigcap \text{Dom}[p]$ characterizes all mandatory root-to- v checkpoints; invert membership counts.

Correctness

After fixed-point convergence, v belongs to $\text{Dom}[x]$ exactly when every root-to- x path contains v . Counting this membership over all targets therefore counts exactly the vertices in v 's dominator-tree subtree, including v itself, without needing to construct the immediate-dominator edges.

Complexity

The same bounded dominator fixed point plus $O(n^2)$ membership aggregation.

C++

```
#include <bits/stdc++.h>
using namespace std;
struct SCC {
    int n, c = 0;
    vector<vector<int>> g, rg;
    vector<int> id, order;
    SCC(int n, vector<pair<int, int>> e) : n(n), g(n), rg(n), id(n, -1) {
        for (auto [u, v] : e)
            g[u].push_back(v), rg[v].push_back(u);
        vector<char> seen(n);
        for (int s = 0; s < n; s++)
            if (!seen[s]) {
                vector<pair<int, int>> st = {{s, 0}};
                seen[s] = 1;
                while (!st.empty()) {
                    auto &[u, i] = st.back();
                    if (i < (int)g[u].size()) {
                        int v = g[u][i++];
                        if (!seen[v])
                            seen[v] = 1, st.push_back({v, 0});
                    } else
                        order.push_back(u), st.pop_back();
                }
            }
        reverse(order.begin(), order.end());
        for (int s : order)
            if (id[s] < 0) {
                id[s] = c;
                vector<int> st = {s};
                while (!st.empty()) {
                    int u = st.back();
                    st.pop_back();
                    for (int v : rg[u])
                        if (id[v] < 0)
                            id[v] = c, st.push_back(v);
                }
            }
    }
};
```

```

        c++;
    }
}
};

struct LowLink {
    int n, timer = 0;
    vector<pair<int, int>> e;
    vector<vector<pair<int, int>>> g;
    vector<int> tin, low, sub, parent_edge, it;
    vector<char> bridge;
    vector<vector<int>> parts;
    LowLink(int n, vector<pair<int, int>> e)
        : n(n), e(e), g(n), tin(n, -1), low(n), sub(n), parent_edge(n, -1), it(n),
          bridge(e.size()), parts(n) {
        for (int i = 0; i < (int)e.size(); i++) {
            auto [u, v] = e[i];
            g[u].push_back({v, i});
            g[v].push_back({u, i});
        }
        tin[0] = low[0] = timer++;
        sub[0] = 1;
        vector<int> st = {0};
        while (!st.empty()) {
            int u = st.back();
            if (it[u] < (int)g[u].size()) {
                auto [v, id] = g[u][it[u]++];
                if (id == parent_edge[u])
                    continue;
                if (tin[v] >= 0)
                    low[u] = min(low[u], tin[v]);
                else {
                    parent_edge[v] = id;
                    tin[v] = low[v] = timer++;
                    sub[v] = 1;
                    st.push_back(v);
                }
            } else {
                st.pop_back();
                int id = parent_edge[u];
                if (id < 0)
                    continue;
                auto [a, b] = e[id];
                int p = a ^ b ^ u;
                sub[p] += sub[u];
                low[p] = min(low[p], low[u]);
                if (low[u] > tin[p])
                    bridge[id] = 1;
                if (low[u] >= tin[p])
                    parts[p].push_back(sub[u]);
            }
        }
    }
};

pair<vector<int>, vector<vector<int>>>
bridge_tree(int n, const vector<pair<int, int>> &e) {
    LowLink l(n, e);
    vector<int> id(n, -1);
    int c = 0;
    for (int s = 0; s < n; s++)
        if (id[s] < 0) {
            id[s] = c;
            vector<int> st = {s};
            while (!st.empty()) {

```

```

        int u = st.back();
        st.pop_back();
        for (auto [v, k] : l.g[u])
            if (!l.bridge[k] && id[v] < 0)
                id[v] = c, st.push_back(v);
    }
    c++;
}
vector<vector<int>> t(c);
for (int i = 0; i < (int)e.size(); i++)
    if (l.bridge[i]) {
        auto [u, v] = e[i];
        u = id[u];
        v = id[v];
        t[u].push_back(v);
        t[v].push_back(u);
    }
return {id, t};
}
struct TreeDistance {
    int n, L;
    vector<int> d;
    vector<vector<int>> up;
    TreeDistance(vector<vector<int>> t)
        : n(t.size()), L(max(1, (int)bit_width((unsigned)max(1, n)))), d(n),
          up(L, vector<int>(n)) {
        vector<pair<int, int>> st = {{0, 0}};
        while (!st.empty()) {
            auto [u, p] = st.back();
            st.pop_back();
            up[0][u] = p;
            for (int v : t[u])
                if (v != p)
                    d[v] = d[u] + 1, st.push_back({v, u});
        }
        for (int j = 1; j < L; j++)
            for (int i = 0; i < n; i++)
                up[j][i] = up[j - 1][up[j - 1][i]];
    }
    int lca(int a, int b) {
        if (d[a] < d[b])
            swap(a, b);
        int z = d[a] - d[b];
        for (int j = 0; j < L; j++)
            if (z >> j & 1)
                a = up[j][a];
        if (a == b)
            return a;
        for (int j = L - 1; j >= 0; j--)
            if (up[j][a] != up[j][b])
                a = up[j][a], b = up[j][b];
        return up[0][a];
    }
    int dist(int a, int b) {
        int c = lca(a, b);
        return d[a] + d[b] - 2 * d[c];
    }
};
vector<vector<int>> block_cut(int n, const vector<pair<int, int>> &e) {
    vector<vector<pair<int, int>>> g(n);
    for (int i = 0; i < (int)e.size(); i++) {
        auto [u, v] = e[i];
        g[u].push_back({v, i});
    }
}

```

```

    g[v].push_back({u, i});
}
vector<int> tin(n, -1), low(n), edges, parent(n, -1), it(n);
vector<vector<int>> t(n);
int timer = 0;
tin[0] = low[0] = timer++;
vector<int> dfs = {0};
while (!dfs.empty()) {
    int u = dfs.back();
    if (it[u] < (int)g[u].size()) {
        auto [v, id] = g[u][it[u]++];
        if (id == parent[u])
            continue;
        if (tin[v] < 0) {
            parent[v] = id;
            edges.push_back(id);
            tin[v] = low[v] = timer++;
            dfs.push_back(v);
        } else if (tin[v] < tin[u])
            edges.push_back(id), low[u] = min(low[u], tin[v]);
    } else {
        dfs.pop_back();
        int id = parent[u];
        if (id < 0)
            continue;
        auto [a, b] = e[id];
        int p = a ^ b ^ u;
        low[p] = min(low[p], low[u]);
        if (low[u] >= tin[p]) {
            vector<int> vs;
            while (1) {
                int x = edges.back();
                edges.pop_back();
                vs.push_back(e[x].first);
                vs.push_back(e[x].second);
                if (x == id)
                    break;
            }
            sort(vs.begin(), vs.end());
            vs.erase(unique(vs.begin(), vs.end()), vs.end());
            int block = t.size();
            t.push_back({});
            for (int x : vs)
                t[block].push_back(x), t[x].push_back(block);
        }
    }
}
return t;
}

vector<bitset<1500>> dominators(int n, const vector<pair<int, int>> &e) {
    vector<vector<int>> g(n), pred(n);
    for (auto [u, v] : e)
        g[u].push_back(v), pred[v].push_back(u);
    bitset<1500> reach;
    reach[0] = 1;
    vector<int> st = {0};
    while (!st.empty()) {
        int u = st.back();
        st.pop_back();
        for (int v : g[u])
            if (!reach[v])
                reach[v] = 1, st.push_back(v);
    }
}

```

```

vector<bitset<1500>> d(n, reach);
d[0].reset();
d[0][0] = 1;
bool change = 1;
while (change) {
    change = 0;
    for (int v = 1; v < n; v++)
        if (reach[v]) {
            bitset<1500> x = reach;
            for (int p : pred[v])
                if (reach[p])
                    x &= d[p];
            x[v] = 1;
            if (x != d[v])
                d[v] = x, change = 1;
        }
    }
    return d;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    vector<pair<int, int>> e(m);
    for (auto &[u, v] : e)
        cin >> u >> v, --u, --v;
    auto d = dominators(n, e);
    vector<int> size(n);
    for (int x = 0; x < n; x++)
        for (int v = 0; v < n; v++)
            size[v] += d[x][v];
    while (q--) {
        int v;
        cin >> v;
        cout << size[v - 1] << '\n';
    }
}

```

Python

```

import sys

sys.setrecursionlimit(1_000_000)

def scc(n, edges):
    graph = [[] for _ in range(n)]
    reverse = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        reverse[v].append(u)
    seen = [False] * n
    order = []
    for start in range(n):
        if seen[start]:
            continue
        stack = [(start, 0)]
        seen[start] = True
        while stack:
            node, index = stack[-1]

```

```

        if index < len(graph[node]):
            target = graph[node][index]
            stack[-1] = (node, index + 1)
            if not seen[target]:
                seen[target] = True
                stack.append((target, 0))
        else:
            order.append(node)
            stack.pop()
    component = [-1] * n
    count = 0
    for start in reversed(order):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for target in reverse[node]:
                if component[target] == -1:
                    component[target] = count
                    stack.append(target)
        count += 1
    return component, count

def undirected_lowlink(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    subtree = [0] * n
    bridges = [False] * len(edges)
    separating = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        subtree[node] = 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:
                continue
            if tin[target] != -1:
                low[node] = min(low[node], tin[target])
            else:
                dfs(target, edge_id)
                subtree[node] += subtree[target]
                low[node] = min(low[node], low[target])
            if low[target] > tin[node]:
                bridges[edge_id] = True
            if low[target] >= tin[node]:
                separating[node].append(subtree[target])

    dfs(0)
    return graph, tin, low, subtree, bridges, separating

def bridge_tree(n, edges):
    graph, _, _, _, bridges, _ = undirected_lowlink(n, edges)

```

```

component = [-1] * n
count = 0
for start in range(n):
    if component[start] != -1:
        continue
    component[start] = count
    stack = [start]
    while stack:
        node = stack.pop()
        for target, edge_id in graph[node]:
            if not bridges[edge_id] and component[target] == -1:
                component[target] = count
                stack.append(target)
    count += 1
tree = [[] for _ in range(count)]
for edge_id, (u, v) in enumerate(edges):
    if bridges[edge_id]:
        a, b = component[u], component[v]
        tree[a].append(b)
        tree[b].append(a)
return component, tree

def tree_lca(tree):
    size = len(tree)
    levels = max(1, size.bit_length())
    up = [[0] * size for _ in range(levels)]
    depth = [0] * size
    stack = [(0, 0)]
    order = [0]
    while stack:
        node, parent = stack.pop()
        up[0][node] = parent
        for target in tree[node]:
            if target != parent:
                depth[target] = depth[node] + 1
                order.append(target)
                stack.append((target, node))
    for level in range(1, levels):
        previous = up[level - 1]
        up[level] = [previous[previous[node]] for node in range(size)]

    def lca(a, b):
        if depth[a] < depth[b]:
            a, b = b, a
        difference = depth[a] - depth[b]
        for level in range(levels):
            if difference >> level & 1:
                a = up[level][a]
        if a == b:
            return a
        for level in range(levels - 1, -1, -1):
            if up[level][a] != up[level][b]:
                a, b = up[level][a], up[level][b]
        return up[0][a]

    def distance(a, b):
        ancestor = lca(a, b)
        return depth[a] + depth[b] - 2 * depth[ancestor]

    return distance

```

```

def block_cut_tree(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    edge_stack = []
    tree = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:
                continue
            if tin[target] == -1:
                edge_stack.append(edge_id)
                dfs(target, edge_id)
                low[node] = min(low[node], low[target])
                if low[target] >= tin[node]:
                    vertices = set()
                    while True:
                        taken = edge_stack.pop()
                        vertices.update(edges[taken])
                        if taken == edge_id:
                            break
                    block = len(tree)
                    tree.append([])
                    for vertex in vertices:
                        tree[block].append(vertex)
                        tree[vertex].append(block)
            elif tin[target] < tin[node]:
                edge_stack.append(edge_id)
                low[node] = min(low[node], tin[target])

    dfs(0)
    return tree

def dominators(n, edges):
    predecessors = [[] for _ in range(n)]
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        predecessors[v].append(u)
    reachable = 1
    stack = [0]
    while stack:
        node = stack.pop()
        for target in graph[node]:
            if not (reachable >> target & 1):
                reachable |= 1 << target
                stack.append(target)
    dom = [reachable] * n
    dom[0] = 1
    changed = True
    while changed:
        changed = False
        for node in range(1, n):
            if not (reachable >> node & 1):

```

```

        continue
    value = reachable
    for parent in predecessors[node]:
        if reachable >> parent & 1:
            value &= dom[parent]
    value |= 1 << node
    if value != dom[node]:
        dom[node] = value
        changed = True
    return dom

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m, q = data[:3]
    at = 3
    edges = [(data[i] - 1, data[i + 1] - 1) for i in range(at, at + 2 * m, 2)]
    at += 2 * m
    dom = dominators(n, edges)
    size = [0] * n
    for target in range(n):
        bits = dom[target]
        while bits:
            bit = bits & -bits
            size[bit.bit_length() - 1] += 1
            bits -= bit
    print("\n".join(str(size[data[i] - 1]) for i in range(at, at + q)))

if __name__ == "__main__":
    main()

```

Contest-wide validation

Use transitive closure for SCC reachability, enumerate candidate repair edges, backtrack over word-edge orders, enumerate every orientation, delete vertices and run BFS, and intersect all tiny root paths for dominators. In particular, test the already-strongly-connected case, disconnected nonzero-degree Euler edges, multigraph bridges, failed endpoints, and unreachable directed vertices.

The transferable habit is to translate a global path quantifier into the correct decomposition object before optimizing queries on it.